GUIA PRÁTICO

Projeto Mural de Recados

JavaScript do Zero

Aprendendo com o Mural de Recados

Variáveis, funções, DOM, eventos e localStorage,
passando pelos erros que todo iniciante enfrenta.

script.js

```
const botao = document.getElementById("envio");

botao.addEventListener("click", function () {
  const nome = campoNome.value.trim();

  if (nome === "") {
    alert("Preencha seu nome!");
  }
});
```

14 CAPÍTULOS · EXEMPLOS · EXERCÍCIOS

Sumário

Como usar este material 3

Capítulo 1: O que é JavaScript e como ligar ao HTML 3

Capítulo 2: Variáveis e tipos 4

Capítulo 3: Operadores 5

Capítulo 4: Tomando decisões com if 6

Capítulo 5: Funções 7

Capítulo 6: Listas (arrays) e objetos 9

Capítulo 7: Repetição 10

Capítulo 8: O DOM, ou como o JS mexe na página 11

Capítulo 9: Eventos 13

Capítulo 10: Erros e depuração 14

Capítulo 11: Guardando dados com localStorage 15

Capítulo 12: O JavaScript completo do Mural 17

Capítulo 13: Exercícios 20

Capítulo 14: Folha de consulta rápida 21

Próximos passos 22

{ }

COMECE AQUI

Como usar este material

Este guia continua o de CSS e usa o mesmo projeto: o **Mural de Recados**. O objetivo é que você entenda o *porquê* de cada linha, e não só copie código.

Sugestão de estudo: leia um capítulo, digite os exemplos (sem copiar e colar), mude os valores e veja o resultado no console do navegador. Errar e ler a mensagem de erro faz parte do aprendizado, e o Capítulo 10 é dedicado a isso.

01

CAPÍTULO 1

O que é JavaScript e como ligar ao HTML

Três linguagens, três funções:

- **HTML**: a estrutura (títulos, campos, botões).
- **CSS**: a aparência (cores, tamanhos, espaços).
- **JavaScript**: o comportamento (o que acontece quando alguém clica, digita, envia).

Para o navegador usar o seu arquivo JS, coloque esta linha no **final do** `<body>`, depois de todo o conteúdo:

HTML

```
    </div>

    <script src="script.js"></script>
</body>
</html>
```

Por que no final? O navegador lê o HTML de cima para baixo. Se o JS rodar antes de o botão existir, ele não vai encontrá-lo e o resultado será `null`. Colocando o script no fim, tudo já foi criado.

O console: seu melhor amigo

Aperte **F12** no navegador e abra a aba **Console**. Para testar se o arquivo está ligado, escreva na primeira linha do `script.js`:

```JavaScript
console.log("script carregou!");
```

Salve, recarregue com **Ctrl + F5** e veja a mensagem aparecer. O `console.log` serve para "espiar" o que o seu código está fazendo.

02 CAPÍTULO 2 Variáveis e tipos

Uma variável é uma "caixinha com nome" que guarda um valor.

```JavaScript
const nome = "Lucas";
let contador = 0;

contador = contador + 1;
```

- `const`: o valor **não será reatribuído**. Use por padrão.
- `let`: o valor **vai mudar** depois (como um contador).
- `var`: forma antiga, tem comportamentos confusos. Evite.

Regras para nomes

- Não podem começar com número nem ter espaços.
- **Maiúsculas e minúsculas fazem diferença**: `campoNome`, `camponome` e `CampoNome` são três variáveis diferentes. Foi exatamente esse o erro `camponome is not defined` que apareceu no seu console.
- O padrão mais usado é o **camelCase**: a primeira palavra em minúsculas e as outras com a inicial maiúscula (`campoNome`, `listaRecados`, `botaoEnvio`).

Tipos de valores

Tipo	Exemplo	Para que serve
string	`"Olá"`	texto
number	`42` , `3.14`	números
boolean	`true` , `false`	verdadeiro ou falso
null	`null`	“vazio de propósito”
undefined	`undefined`	“nada foi definido ainda”

Para descobrir o tipo de algo, use `typeof` :

```JavaScript
console.log(typeof "oi");   // string
console.log(typeof 10);     // number
```

03

CAPÍTULO 3

Operadores

Matemática

```JavaScript
5 + 3   // 8
5 - 3   // 2
5 * 3   // 15
5 / 2   // 2.5
5 % 2   // 1 (resto da divisão)
```

Juntando textos

```JavaScript
const hora = "14:35";

console.log("Hoje às " + hora);     // com +
console.log(`Hoje às ${hora}`);      // com crases (template string)
```

As **crases** (`` ` ``) permitem colocar variáveis dentro do texto com `${...}`. É mais limpo quando há várias partes.

Comparações

```JavaScript
5 === 5     // true  (igual)
5 !== 3     // true  (diferente)
5 > 3       // true
5 <= 3      // false
```

Use sempre **três sinais** (`===`) para comparar. O `==` converte tipos escondido e dá resultados surpreendentes.

> **Cuidado:** `=` (um sinal) **guarda** um valor; `===` **compara**.

Operadores lógicos

Operador	Significa	Exemplo
`&&`	E (os dois precisam ser verdadeiros)	`idade > 18 && temCadastro`
`||`	OU (basta um ser verdadeiro)	`nome === "" || texto === ""`
`!`	NÃO (inverte)	`!logado`

CAPÍTULO 4

04 Tomando decisões com if

```JavaScript
const idade = 20;

if (idade >= 18) {
  console.log("Maior de idade");
} else if (idade >= 16) {
  console.log("Pode votar");
} else {
  console.log("Menor de idade");
}
```

A leitura é: “**se** a condição for verdadeira, faça isto; **senão**, se esta outra for, faça aquilo; **senão**, faça isto”.

Exemplo do Mural, validando os campos:

```JavaScript
if (nome === "" || texto === "") {
  alert("Preencha seu nome e o aviso!");
}
```

Valores “falsy”

Estes valores contam como falso dentro de um `if`: `""` (texto vazio), `0`, `null`, `undefined` e `false`. Por isso, a validação acima também poderia ser escrita assim:

```JavaScript
if (!nome || !texto) {
  alert("Preencha seu nome e o aviso!");
}
```

05

CAPÍTULO 5

Funções

Uma função é um bloco de código com nome, que você pode usar quantas vezes quiser.

```JavaScript
function saudar(nome) {
  return "Olá, " + nome + "!";
}

const mensagem = saudar("Lucas");
console.log(mensagem);   // Olá, Lucas!
```

- `function saudar(nome)`: cria a função. `nome` é o **parâmetro** (o que ela recebe).
- `return`: o valor que ela **devolve**. Depois do `return`, a função termina.
- `saudar("Lucas")`: **chama** a função, passando o argumento `"Lucas"`.

Funções sem nome e arrow functions

No Mural você usou uma função “solta” dentro do `addEventListener`:

```JavaScript
botao.addEventListener("click", function () {
  console.log("Clicou!");
});
```

Existe uma forma mais curta, a **arrow function**, que faz o mesmo:

```JavaScript
botao.addEventListener("click", () => {
  console.log("Clicou!");
});
```

Escopo: onde cada variável “vive”

Uma variável criada **dentro** de uma função só existe ali dentro.

```JavaScript
function teste() {
  const segredo = 123;
}

teste();
console.log(segredo);   // ReferenceError: segredo is not defined
```

O erro clássico: código fora da função

Este foi o problema no seu `script.js`:

```JavaScript
botao.addEventListener("click", function () {
  const nome = campoNome.value;
});                                   // a função termina aqui

if (nome === "") {                    // nome não existe aqui fora
  return;                             // return fora de função = erro de sintaxe
}
```

Dois problemas ao mesmo tempo:

1. `nome` só existe dentro da função, então o `if` de fora não a enxerga.
2. `return` só pode ser usado **dentro** de uma função. Fora dela, dá `Illegal return statement`, e o navegador **não executa o arquivo inteiro**.

A correção é colocar o `if` dentro das chaves da função.

Chaves e indentação

Toda `{` precisa de uma `}` correspondente. Para não se perder, **indente** o código (dois espaços) cada vez que abrir uma chave. No VS Code, o atalho **Shift + Alt + F** formata tudo automaticamente e deixa visível quando uma chave está no lugar errado.

06

CAPÍTULO 6

Listas (arrays) e objetos

Arrays

Uma lista de valores. A contagem começa no **zero**.

JavaScript

```javascript
const frutas = ["maçã", "banana", "uva"];

console.log(frutas[0]);       // maçã
console.log(frutas.length);   // 3

frutas.push("pera");          // adiciona no final
```

Objetos

Um “cadastro” com várias informações, cada uma com um nome (chave):

JavaScript

```javascript
const recado = {
  nome: "Lucas",
  texto: "Bem-vindos!",
  data: "17/10/2026"
};

console.log(recado.nome);     // Lucas
recado.texto = "Novo texto";  // alterar
```

Lista de objetos

É assim que o Mural guarda os recados:

```JavaScript
const recados = [
  { nome: "Lucas", texto: "Oi!", data: "17/10/2026" },
  { nome: "Ana", texto: "Olá!", data: "17/10/2026" }
];

recados.push({ nome: "Beto", texto: "E aí!", data: "18/10/2026" });
```

filter: criar uma lista nova sem alguns itens

```JavaScript
const numeros = [1, 2, 3, 4, 5];
const pares = numeros.filter(function (n) {
  return n % 2 === 0;
});
console.log(pares);   // [2, 4]
```

O `filter` mantém só os itens para os quais a função devolve `true`. Vamos usá-lo para excluir recados.

07

CAPÍTULO 7

Repetição

for clássico

```JavaScript
for (let i = 0; i < 3; i++) {
  console.log("Volta número", i);
}
```

- `let i = 0`: começa em 0.
- `i < 3`: continua enquanto for menor que 3.
- `i++`: soma 1 a cada volta.

forEach

Para percorrer uma lista, o `forEach` é mais simples:

```JavaScript
const frutas = ["maçã", "banana", "uva"];

frutas.forEach(function (fruta) {
  console.log(fruta);
});
```

No Mural, ele desenha cada recado guardado ao abrir a página.

08 CAPÍTULO 8 O DOM, ou como o JS mexe na página

O navegador transforma o seu HTML em uma árvore de objetos chamada **DOM**. O JavaScript consegue ler e alterar essa árvore.

Encontrando elementos

```JavaScript
const campoNome = document.getElementById("nome");
const primeiroRecado = document.querySelector(".recado");
const todosRecados = document.querySelectorAll(".recado");
```

- `getElementById` : busca pelo `id` .
- `querySelector` : usa um **seletor CSS** (`.classe` , `#id` , `tag`) e devolve o primeiro que encontrar.
- `querySelectorAll` : devolve todos.

Se o elemento não for encontrado, o resultado é `null` , e usar algo como `.addEventListener` nele gera o erro `Cannot read properties of null` .

Lendo e escrevendo conteúdo

```JavaScript
campoNome.value                // lê o que foi digitado
campoNome.value = "";          // limpa o campo

titulo.textContent = "Novo";   // troca o texto de um elemento
```

- `.value` : para campos de formulário (`input` , `textarea`).
- `.textContent` : para o texto de elementos comuns (`p` , `span` , `h1`).
- `.trim()` remove espaços do começo e do fim: `campoNome.value.trim()` .

> **Sobre** `innerHTML` **:** ele interpreta o texto como HTML, e se alguém digitar código malicioso no campo, ele seria executado na página. Para conteúdo vindo de usuários, use `textContent` .

Classes

```JavaScript
elemento.classList.add("destaque");
elemento.classList.remove("destaque");
elemento.classList.toggle("destaque");   // liga se estiver desligada, desliga se estiver l
igada
```

Esta é a ponte entre JS e CSS: o JS troca a classe, e o CSS cuida do visual.

Criando e removendo elementos

```JavaScript
const recado = document.createElement("div");   // cria na memória
recado.classList.add("recado");
recado.textContent = "Meu recado";

listaRecados.append(recado);    // coloca no final da lista
listaRecados.prepend(recado);   // coloca no topo da lista

recado.remove();                // tira da página
```

Um elemento criado com `createElement` **não aparece** até você inseri-lo na página com `append` ou `prepend` .

09

CAPÍTULO 9

Eventos

Um evento é algo que acontece na página: um clique, uma tecla, o envio de um formulário.

```JavaScript
elemento.addEventListener("evento", funcao);
```

Eventos mais usados

Evento	Quando acontece
`click`	clique do mouse
`submit`	envio de um formulário
`input`	a cada alteração em um campo
`keydown`	ao apertar uma tecla
`change`	ao confirmar a alteração de um campo

Evitando o recarregamento do formulário

Se você apertar **Enter** dentro de um campo, o formulário é enviado e a página recarrega (por isso apareceu `index.html?` na sua barra de endereço). Para cancelar:

```JavaScript
formulario.addEventListener("submit", function (evento) {
  evento.preventDefault();
});
```

A função recebe um parâmetro (`evento`) com informações do que aconteceu, e `preventDefault()` cancela o comportamento padrão.

Exemplo: contador de caracteres

```HTML
<span id="contador">0/200</span>
```

```JavaScript
const contador = document.getElementById("contador");

campoAviso.addEventListener("input", function () {
  contador.textContent = campoAviso.value.length + "/200";
});
```

CAPÍTULO 10

10 Erros e depuração

Erros são normais, e o console em vermelho **diz o que e onde** deu errado. Aprenda a ler:

```Console
Uncaught ReferenceError: camponome is not defined
    at script.js:14
```

- **O tipo do erro**: `ReferenceError`.
- **A explicação**: `camponome is not defined`.
- **O lugar**: arquivo `script.js`, linha 14.

Os erros mais comuns

Mensagem	O que significa	Causas comuns
`X is not defined`	a variável não existe	nome digitado errado (maiúsculas!), variável fora do escopo, script não carregado
`Cannot read properties of null`	o elemento não foi encontrado	`id` diferente no HTML, `<script>` antes do elemento
`Illegal return statement`	`return` fora de uma função	`if` com `return` ficou fora das chaves da função
`Unexpected token` ou `Unexpected end of input`	falta ou sobra algo	chave, parêntese ou aspas sem fechar
`X is not a function`	você chamou algo que não é função	erro de digitação em um método (`addeventlistener`)

Passo a passo para achar um bug

1. Recarregue com Ctrl + F5 e abra o console.
2. Leia o **primeiro** erro vermelho (os de baixo costumam ser consequência dele).
3. Vá até a linha indicada.
4. Se não houver erro, espalhe `console.log` pelo código para ver até onde ele chega:

```JavaScript
console.log("1: entrou no clique");
console.log("2: nome =", nome);
```

5. Isole o problema: apague tudo e teste só o pedaço mínimo (por exemplo, apenas o clique do botão).

Erros de sintaxe derrubam o arquivo todo

Quando existe um erro de **sintaxe** (chave faltando, `return` fora de função), o navegador nem começa a executar o arquivo. Por isso, às vezes um botão “não faz nada” mesmo com o `addEventListener` correto: o erro estava em outra parte do arquivo.

11 CAPÍTULO 11 Guardando dados com localStorage

Sem isso, os recados somem ao recarregar a página, porque o JS só altera a memória. O `localStorage` guarda dados **dentro do navegador**.

```JavaScript
localStorage.setItem("chave", "valor");    // guardar
localStorage.getItem("chave");             // ler
localStorage.removeItem("chave");          // apagar uma chave
localStorage.clear();                      // apagar tudo
```

Só guarda texto, então use JSON

Listas e objetos precisam virar texto antes de salvar, e voltar a ser lista ao ler:

```javascript
const lista = [{ nome: "Ana" }];

localStorage.setItem("recados", JSON.stringify(lista));

const texto = localStorage.getItem("recados");
const voltou = texto ? JSON.parse(texto) : [];
```

- `JSON.stringify` : objeto/lista para texto.
- `JSON.parse` : texto para objeto/lista.
- `texto ? ... : []` significa: “se existir algo guardado, converta; senão, use uma lista vazia”.

Como conferir

Aperte F12, vá na aba **Application**, depois **Local Storage**. Lá você vê as chaves guardadas e pode apagar na mão.

> **Limite importante:** os dados ficam só no seu navegador, neste computador. Outras pessoas não veem os seus recados. Para um mural realmente compartilhado, seria necessário um servidor com banco de dados.

12

CAPÍTULO 12

O JavaScript completo do Mural

Juntando tudo, com `localStorage` e botão de excluir. Cada recado ganha um `id` único (`Date.now()`) para saber qual remover.

JavaScript

```
const campoNome = document.getElementById("nome");
const campoAviso = document.getElementById("aviso");
const botaoEnvio = document.getElementById("envio");
const listaRecados = document.getElementById("lista-recados");
const formulario = document.getElementById("form_principal");

const CHAVE = "recados";

function carregarRecados() {
  const salvo = localStorage.getItem(CHAVE);
  return salvo ? JSON.parse(salvo) : [];
}

let recados = carregarRecados();

function salvarRecados() {
  localStorage.setItem(CHAVE, JSON.stringify(recados));
}

function criarRecado(recado) {
  const caixa = document.createElement("div");
  caixa.classList.add("recado");

  const autor = document.createElement("strong");
  autor.textContent = recado.nome;

  const data = document.createElement("span");
  data.classList.add("data");
  data.textContent = recado.data;

  const conteudo = document.createElement("p");
  conteudo.classList.add("recado-texto");
  conteudo.textContent = recado.texto;

  const botaoExcluir = document.createElement("button");
  botaoExcluir.type = "button";
  botaoExcluir.classList.add("btn-excluir");
  botaoExcluir.textContent = "Excluir";

  botaoExcluir.addEventListener("click", function () {
    recados = recados.filter(function (r) {
      return r.id !== recado.id;
    });
    salvarRecados();
    caixa.remove();
  });
```

```
  caixa.append(autor, document.createElement("br"), data, conteudo, botaoExcluir);
  return caixa;
}

// Ao abrir a página, desenha os recados guardados
recados.forEach(function (r) {
  listaRecados.prepend(criarRecado(r));
});

// Evita recarregar a página ao apertar Enter
formulario.addEventListener("submit", function (evento) {
  evento.preventDefault();
});

// Ao clicar em enviar
botaoEnvio.addEventListener("click", function () {
  const nome = campoNome.value.trim();
  const texto = campoAviso.value.trim();

  if (nome === "" || texto === "") {
    alert("Preencha seu nome e o aviso!");
    return;
  }

  const novo = {
    id: Date.now(),
    nome: nome,
    texto: texto,
    data: new Date().toLocaleString("pt-BR", {
      dateStyle: "short",
      timeStyle: "short"
    })
  };

  recados.push(novo);
  salvarRecados();
  listaRecados.prepend(criarRecado(novo));

  campoNome.value = "";
  campoAviso.value = "";
});
```

E um CSS simples para o botão de excluir:

```css
.btn-excluir {
  margin-top: 10px;
  background: #e5484d;
  color: white;
  border: none;
  padding: 6px 12px;
  border-radius: 6px;
  cursor: pointer;
}

.btn-excluir:hover {
  background: #c53a3f;
}
```

Lendo o código de cima para baixo

1. **Pegamos os elementos** da página (cinco `getElementById`).
2. **Definimos as funções** que carregam, salvam e desenham recados.
3. **Desenhamos** os recados guardados ao abrir a página.
4. **Ligamos os eventos**: o `submit` (cancelado) e o `click` do botão.
5. No clique: **lemos, validamos, criamos o objeto, guardamos, desenhamos e limpamos os campos**.

> **Atenção:** se você já testou uma versão anterior que guardava recados sem `id`, rode `localStorage.clear()` no console e recarregue, para começar do zero.

13

CAPÍTULO 13

Exercícios

Tente fazer sem olhar as respostas acima.

1. Crie uma variável com o seu nome e escreva no console `Olá, <seu nome>!` usando template string.
2. Escreva uma função `dobro(n)` que devolve o número multiplicado por 2.
3. Crie uma lista com cinco números e use `forEach` para mostrar cada um no console.
4. Use `filter` para guardar só os números maiores que 10 de uma lista.
5. Faça um botão que, ao ser clicado, troque o texto de um parágrafo.
6. Faça um botão “Modo escuro” que use `classList.toggle("escuro")` no `body`, e crie a classe `.escuro` no CSS.
7. Adicione ao Mural o contador de caracteres do Capítulo 9 e limite o texto a 200 caracteres com o atributo `maxlength` no `textarea`.
8. Desafio: faça os recados mais novos ficarem no topo e mostre uma mensagem “Nenhum recado ainda” quando a lista estiver vazia.

14 CAPÍTULO 14 Folha de consulta rápida

Quero...	Use
guardar um valor que não muda	`const x = ...`
guardar um valor que muda	`let x = ...`
espiar valores	`console.log(x)`
comparar	`===` , `!==` , `>` , `<`
E / OU / NÃO	`&&` , `||` , `!`
decidir	`if / else if / else`
criar uma função	`function nome(parametro) { ... }`
devolver um valor	`return` (só dentro de função)
adicionar a uma lista	`lista.push(item)`
percorrer uma lista	`lista.forEach(function (item) { ... })`
filtrar uma lista	`lista.filter(function (item) { return ... })`
achar um elemento	`getElementById` , `querySelector`
ler um campo	`campo.value`
trocar um texto	`elemento.textContent = ...`
mexer em classes	`classList.add / remove / toggle`
criar um elemento	`document.createElement("div")`
colocar na página	`append` , `prepend`
tirar da página	`elemento.remove()`
reagir a um evento	`addEventListener("click", ...)`
cancelar o envio do formulário	`evento.preventDefault()`
guardar no navegador	`localStorage.setItem` + `JSON.stringify`
ler do navegador	`localStorage.getItem` + `JSON.parse`

>_

PARA CONTINUAR

Próximos passos

Depois de dominar este material:

1. **Responsividade no CSS**: `@media (max-width: 600px)` para adaptar o Mural ao celular.
2. **Editar recados**: um botão que transforma o texto em um campo editável.
3. **Publicar online** de graça no GitHub Pages.
4. **Buscar dados de uma API** com `fetch`, para mostrar informações vindas da internet.
5. **Servidor e banco de dados** (Node.js, por exemplo), para os recados serem realmente compartilhados entre pessoas.

Bons estudos!